

AXI-Lite FIR DSP Peripheral

Modular VHDL IP core implementing a register-controlled
FIR DSP interface with scalable architecture.

Verification results

Author: Gabriela Cabrera
Github: @MGabrielaCabrera
Date: March 2026
Version: 1.0

1.Verification description

This document presents the results of the initial verification phase for the VHDL modules developed in this project. The primary objective of this phase is to perform a first functional validation of each module individually through simulation.

The verification has been carried out using VHDL testbenches that rely on assertion-based checks to validate the expected behavior of the design. These assertions allow the simulation to automatically detect functional mismatches and ensure that the implemented logic behaves according to the intended specification.

For this first verification stage, each module has been tested independently. The document includes simulation waveform captures and relevant observations for each test case in order to illustrate the correct operation of the modules under different conditions.

It should be noted that this verification stage focuses on basic functional validation. In a later phase of the project, the verification environment will be extended using more advanced methodologies, including SystemVerilog-based testbenches, functional coverage, and a more structured verification framework. These improvements will enable a more comprehensive validation of the design and increase confidence in the correctness and robustness of the implementation.

2.AXI-Lite Slave Interface Verification

Module Description

The AXI-Lite Slave Interface module implements a memory-mapped interface compliant with the AXI4-Lite protocol. Its main function is to translate AXI-Lite bus transactions into a simplified internal register interface used by the rest of the design.

The module handles both read and write transactions through the standard AXI-Lite channels: write address, write data, write response, read address, and read data. Internally, the interface generates control signals such as register write enable, register read enable, register addresses, and write data for the connected register file.

Additionally, the module performs basic protocol checks, including address alignment verification and address range validation. When an invalid or misaligned address is detected, the module returns an AXI-Lite slave error response (SLVERR).

Testbench Architecture

The verification of the AXI-Lite Slave Interface module is performed using a dedicated VHDL testbench. The testbench instantiates the design under test (DUT) and emulates the behavior of an AXI-Lite master.

A clock generator provides the system clock with a period of 10 ns, while a reset sequence initializes the DUT before the start of the tests. The testbench includes a simple register file model composed of six 32-bit registers representing the memory-mapped registers accessible through the AXI-Lite interface.

The register file reacts to the internal control signals generated by the DUT:

- `reg_write_en` triggers register updates.
- `reg_read_en` enables register read operations.

Two procedures are implemented to simplify the stimulus generation:

- `axi_write`: performs a complete AXI-Lite write transaction including address, data, and response handshake.
- `axi_read`: performs a complete AXI-Lite read transaction including address and read data handshake.

These procedures allow the testbench to generate multiple test scenarios while respecting the AXI-Lite handshake protocol.

Test Scenarios

A set of directed tests has been implemented using assertion-based verification. Each test stimulates a specific aspect of the AXI-Lite protocol behavior and checks the correctness of the module using VHDL assertions.

The following test scenarios are executed:

1. **Basic Write Transaction:** A write operation is performed to register 0. The test verifies that the write response is OKAY and that the register file is updated with the expected value.
2. **Write to Another Valid Register:** A write operation is issued to register 1 to confirm correct address decoding and register update behavior.
3. **Read Transaction Verification:** The value previously written to register 0 is read back and compared with the expected data.
4. **Read from Another Register:** A read operation is performed on register 1 to verify correct readback functionality.
5. **Partial Write Using Byte Strokes:** A write transaction using partial byte enables (`'WSTRB'`) verifies that only selected bytes of the register are updated.

6. **Misaligned Write Address:** A write transaction is attempted using a misaligned address. The expected response is an AXI-Lite slave error (`SLVERR`).
7. **Out-of-Range Write Address:** A write is issued to an address outside the valid register range. The module should return a `SLVERR` response.
8. **Misaligned Read Address:** A read transaction using a misaligned address verifies that the module detects alignment errors.
9. **Out-of-Range Read Address:** A read operation targeting an invalid address verifies correct error handling.
10. **Back-to-Back Write Transactions:** Multiple consecutive writes are issued to different registers to verify correct operation under continuous traffic.
11. **Write Followed by Immediate Read:** A write operation is immediately followed by a read to confirm that the updated data can be retrieved correctly.

Simulation Results

The simulation results confirm the correct functional behavior of the AXI-Lite Slave Interface module for all tested scenarios.

All transactions follow the expected AXI-Lite handshake protocol. Valid write transactions correctly update the internal register file, and subsequent read operations return the expected data values. The byte strobe functionality correctly enables partial register updates according to the specified write mask.

Error handling mechanisms were also verified. Misaligned and out-of-range addresses correctly trigger AXI-Lite slave error responses (`SLVERR`), demonstrating that the implemented address validation logic functions as intended.

The accompanying waveform captures illustrate representative transactions, including valid read and write operations, byte strobe behavior, and error responses for invalid accesses.

```
Analyzing design files
Elaborating axi_lite_slave_if testbench
Running axi_lite_slave_if_tb testbench. Waveforms saved to waves/waves_axi_lite_slave_if_tb.fst
axi_lite_slave_if_tb.vhd:231:9:@50ns:(report note): Test 1: Write to register 0 and verify response
axi_lite_slave_if_tb.vhd:241:9:@90ns:(report note): Test 2: Write to register 1 and verify response
axi_lite_slave_if_tb.vhd:251:9:@140ns:(report note): Test 3: Read back register 0 and verify data
axi_lite_slave_if_tb.vhd:262:9:@220ns:(report note): Test 4: Read back register 1 and verify data
axi_lite_slave_if_tb.vhd:275:9:@300ns:(report note): Test 5: Write with partial byte strobes (upper two bytes only)
axi_lite_slave_if_tb.vhd:285:9:@400ns:(report note): Test 6: Write to misaligned address -> expect SLVERR
axi_lite_slave_if_tb.vhd:293:9:@450ns:(report note): Test 7: Write to out-of-range address -> expect SLVERR
axi_lite_slave_if_tb.vhd:301:9:@500ns:(report note): Test 8: Read from misaligned address -> expect SLVERR
axi_lite_slave_if_tb.vhd:309:9:@580ns:(report note): Test 9: Read from out-of-range address -> expect SLVERR
axi_lite_slave_if_tb.vhd:317:9:@660ns:(report note): Test 10: Back-to-back writes to consecutive registers
axi_lite_slave_if_tb.vhd:331:9:@810ns:(report note): Test 11: Write then immediate read (reg 3)
axi_lite_slave_if_tb.vhd:342:9:@990ns:(assertion note): End of simulation - all tests passed
Opening waveform viewer (only if the fst file exists)
Test complete!
```

Figure 1 – Test results

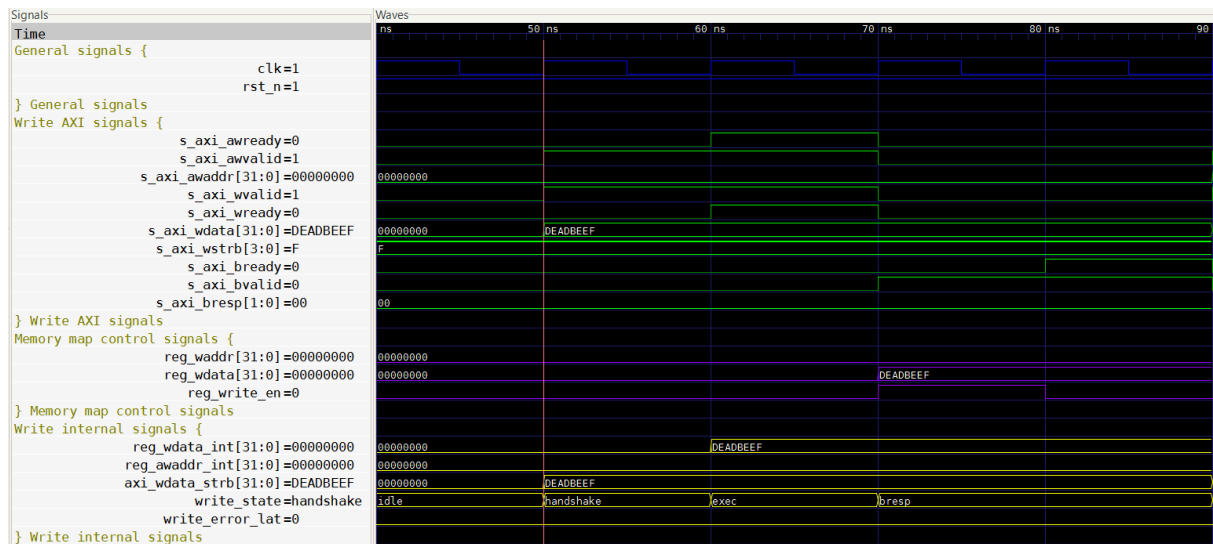


Figure 2 – Test 1: AXI-Lite Write Transaction (Register 0)

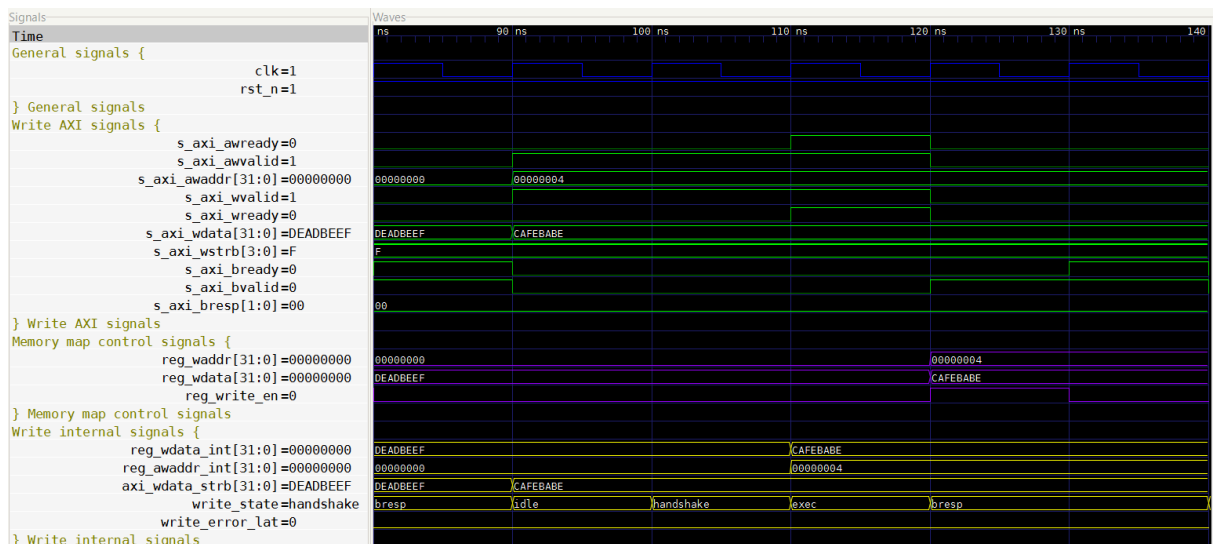


Figure 3 – Test 2: AXI-Lite Write Transaction (Register 1)

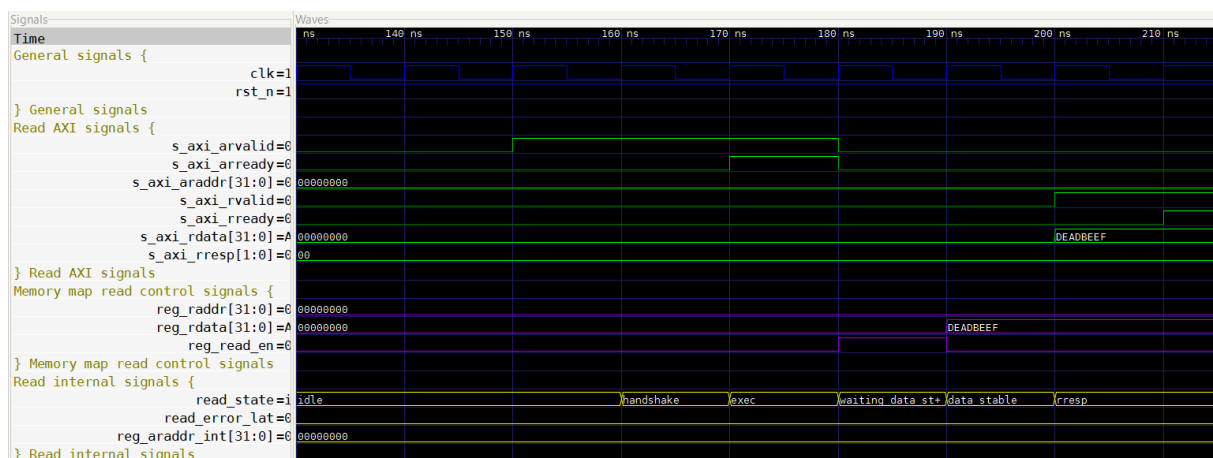
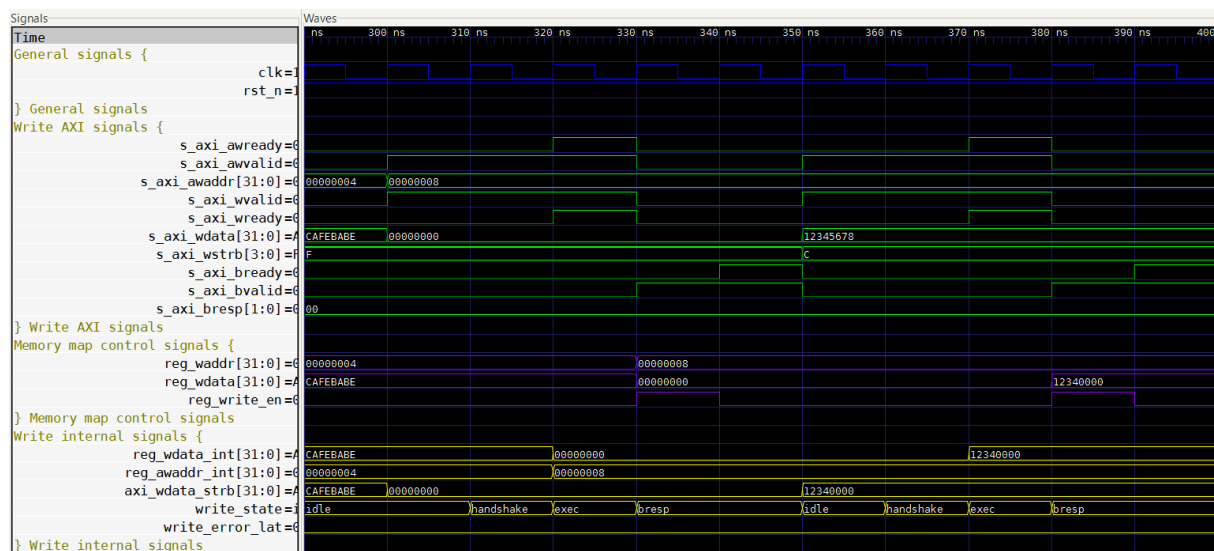
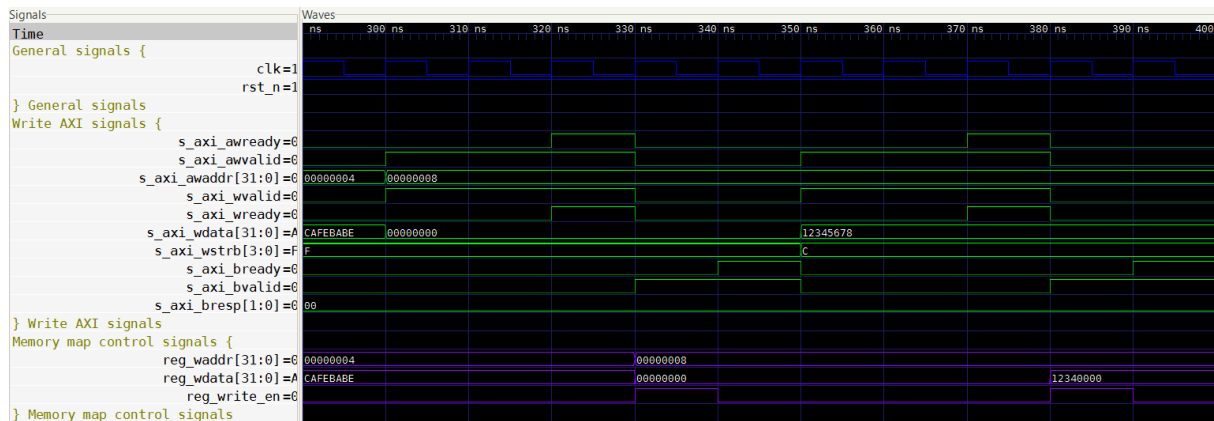
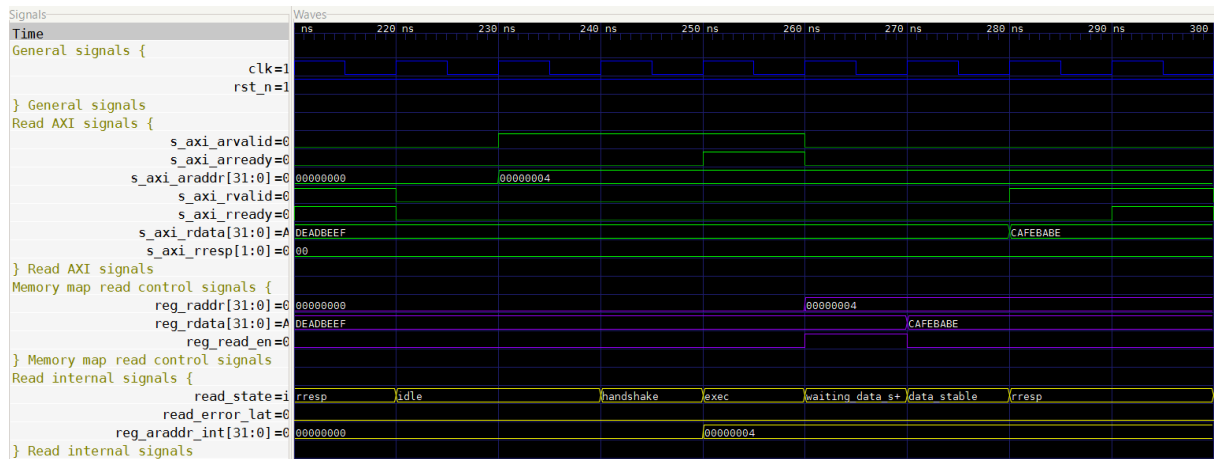
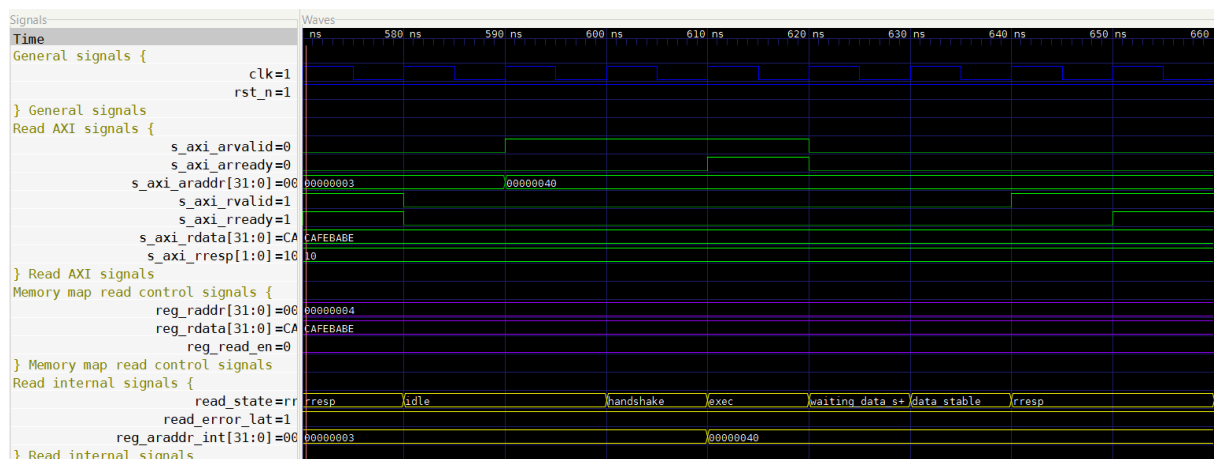
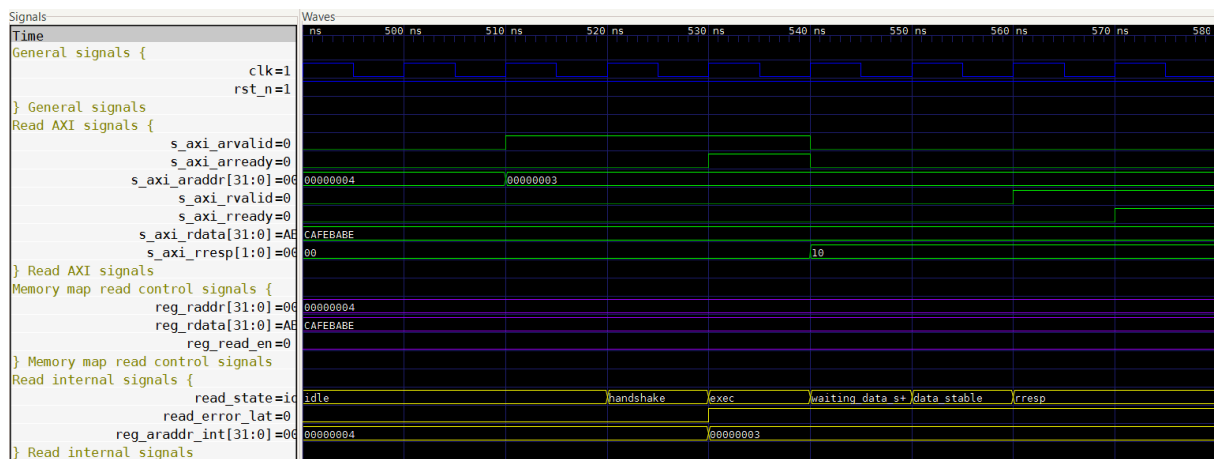
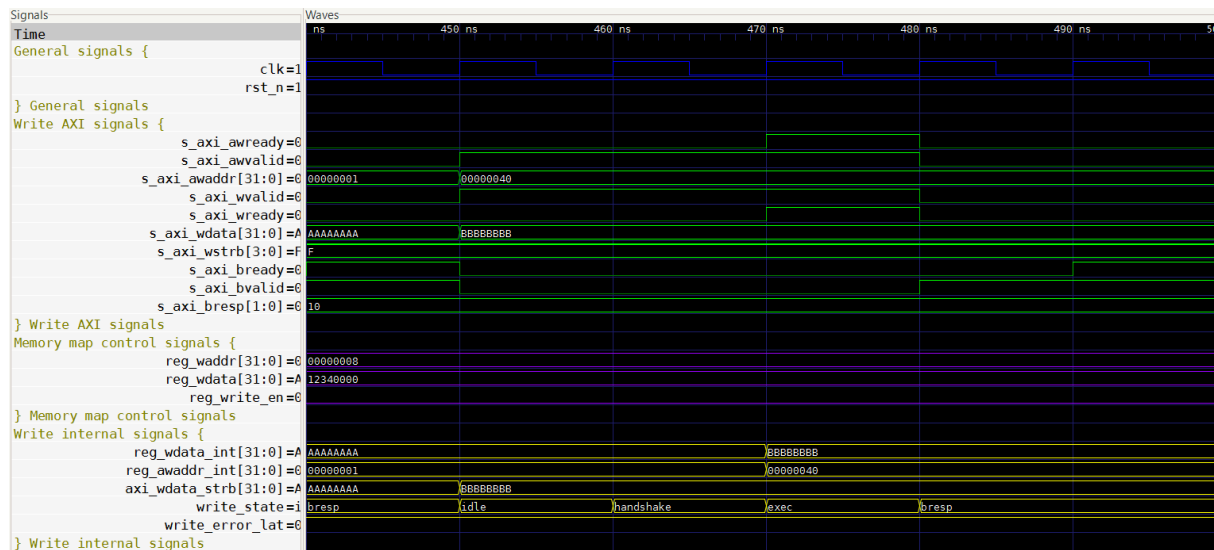
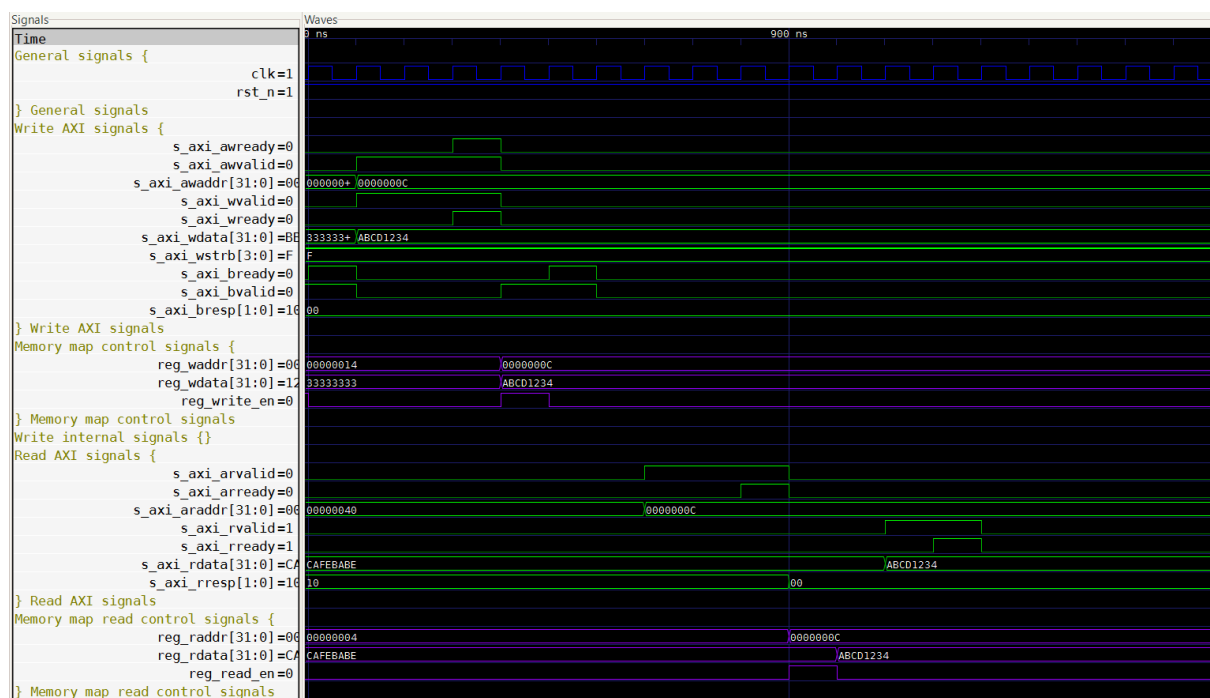
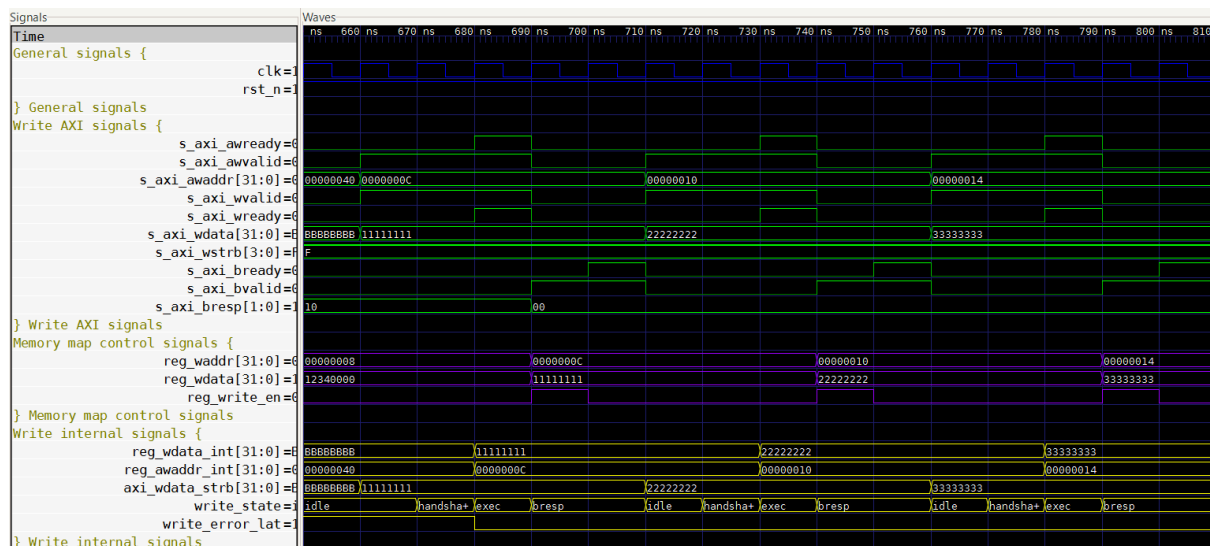


Figure 4 – Test 3: AXI-Lite Read Transaction (Register 0)







3. Control FSM Verification

Module Description

The Control FSM module implements the control logic responsible for coordinating communication between the memory-mapped register interface and an external FIR DSP processing module.

The module operates as a finite state machine that manages two main types of transactions: coefficient loading and sample processing. Input samples and configuration data are received through a register interface and forwarded to the DSP through streaming-style handshake signals.

The FSM includes four operational states:

- IDLE – default state where the module waits for new commands.
- SEND – an input sample is sent to the DSP.
- WAIT_RESULT – the FSM waits for the DSP to produce a processed output sample.
- LOAD_COEFF – a coefficient value is written to the DSP coefficient interface.

The module also generates status information through the `reg_status` signal, indicating whether the module is ready, busy, or in an error condition. Control signals such as `dsp_enable`, `dsp_reset`, and `dsp_mode` are passed through directly from the register interface to the DSP.

The FSM ensures correct sequencing of operations, manages handshake signals for streaming data transfers, and prevents invalid operations such as sending samples when the DSP is disabled or attempting to update coefficients while the DSP is active.

Testbench Architecture

The verification of the Control FSM module is performed using a dedicated VHDL testbench. The testbench instantiates the design under test (DUT) and drives the register interface and DSP handshake signals to emulate the behavior of the surrounding system.

A clock generator produces the system clock with a period of 10 ns, while a reset sequence initializes the FSM before the execution of the verification scenarios.

The testbench provides stimulus to both sides of the interface:

- The register interface, which controls the module through signals such as `reg_enable`, `reg_mode`, `reg_data_in`, `sample_write_strobe`, and `coeff_write_strobe`.
- The DSP interface, which emulates the behavior of the external DSP through signals such as `dsp_data_in_ready`, `dsp_data_out_valid`, and `dsp_data_out`.

Two helper procedures are used to simplify test stimulus generation:

- `clk_rise` – waits for a rising clock edge.
- `clk_fall` – waits for a falling clock edge.

These procedures ensure that inputs are applied during the stable half of the clock cycle so that the DUT samples them correctly at the next rising edge.

The testbench relies on assertion-based verification, where VHDL assertions are used to check that the outputs and state transitions of the FSM match the expected behavior after each stimulus.

Test Scenarios

A set of directed tests has been implemented to validate the functional behavior of the FSM under different operating conditions.

The following verification scenarios are executed:

1. **Reset State:** The module is held in reset and all outputs are verified to remain in their safe default values. This ensures the FSM initializes correctly and no unintended control signals are asserted.
2. **Passthrough Signal Verification:** The signals `dsp_enable`, `dsp_reset`, and `dsp_mode` are verified to correctly mirror the corresponding register values. The test confirms that these signals update on the next clock cycle regardless of the FSM state.
3. **Valid Coefficient Write:** A coefficient write request is issued while the DSP is disabled. The test verifies that the FSM transitions from IDLE to `LOAD_COEFF`, generates a single-cycle write enable pulse, and forwards the correct coefficient address and data.
4. **Blocked Coefficient Write:** A coefficient write request is attempted while the DSP is enabled. The FSM must reject the operation, remain in IDLE, and report an error condition.
5. **Full Data Path Transaction:** A complete data processing transaction is executed. An input sample is sent to the DSP, the FSM waits for the result, and the processed output is captured and forwarded to the register interface. Handshake signals and status updates are verified during the process.
6. **Blocked Sample Transmission:** A sample write request is issued while the DSP is disabled. The FSM must reject the operation and generate an error status without initiating a data transfer.
7. **WAIT_RESULT State Hold:** The FSM is forced into the `WAIT_RESULT` state and the DSP output valid signal is intentionally delayed. The test verifies that the FSM remains in the waiting state and maintains the busy status until a valid result is received.

8. **Output Ready Pulse Width:** The dsp_data_out_ready signal is verified to produce a single clock-cycle pulse when capturing a DSP result, even if the DSP keeps the valid signal asserted for multiple cycles.
9. **Mode Update During Active Transaction:** Changes to the reg_mode register are applied while the FSM is processing a transaction. The test verifies that the updated mode propagates correctly to the dsp_mode signal without interfering with the FSM operation.
10. **Back-to-Back Coefficient Writes:** Two consecutive coefficient write operations are issued to verify that the FSM can correctly handle sequential transactions. Each operation must generate an independent write enable pulse and the FSM must return to the IDLE state between writes.

Simulation Results

The simulation results confirm the correct functional behavior of the Control FSM module across all tested scenarios.

The FSM transitions correctly between the IDLE, SEND, WAIT_RESULT, and LOAD_COEFF states according to the applied stimuli. Streaming handshake signals between the control logic and the DSP are generated with the correct timing and remain stable until the handshake is completed.

Coefficient write operations are correctly executed only when the DSP is disabled, while invalid operations are properly rejected and reported through the status signal. The data path verification confirms that input samples are correctly forwarded to the DSP and that the resulting processed data is captured and stored in the register interface.

Additional tests verify correct handling of extended wait periods, single-cycle handshake pulses, dynamic mode updates, and consecutive coefficient writes.

The accompanying waveform captures illustrate representative scenarios including coefficient loading, sample processing transactions, waiting for DSP results, and error handling conditions.

```
control_fsm_tb.vhd:178:9:@0ms:(report note): Test 1: Verifying output values during active reset (rst_n=0)
control_fsm_tb.vhd:223:9:@50ns:(report note): Test 2: Verifying passthrough signals mirror reg_enable/reg_reset/reg_mode
control_fsm_tb.vhd:263:9:@100ns:(report note): Test 3: Verifying coefficient write with enable=0 (valid path)
control_fsm_tb.vhd:296:9:@125ns:(report note): Test 4: Verifying coefficient write is blocked when enable=1
control_fsm_tb.vhd:332:9:@170ns:(report note): Test 5: Verifying full data path (enable=1, send sample, receive result)
control_fsm_tb.vhd:412:9:@270ns:(report note): Test 6: Verifying sample write is blocked when enable=0
control_fsm_tb.vhd:438:9:@300ns:(report note): Test 7: Verifying FSM stays in WAIT_RESULT until result is valid
control_fsm_tb.vhd:489:9:@400ns:(report note): Test 8: Verifying dsp_data_out_ready is exactly one cycle wide
control_fsm_tb.vhd:530:9:@470ns:(report note): Test 9: Verifying dsp_mode passthrough updates during SEND state
control_fsm_tb.vhd:582:9:@570ns:(report note): Test 10: Verifying back-to-back coefficient writes
control_fsm_tb.vhd:641:9:@660ns:(report note): End of simulation - all tests passed
Opening waveform viewer (only if the fst file exists)
Test complete!
```

Figure 13 – Test results

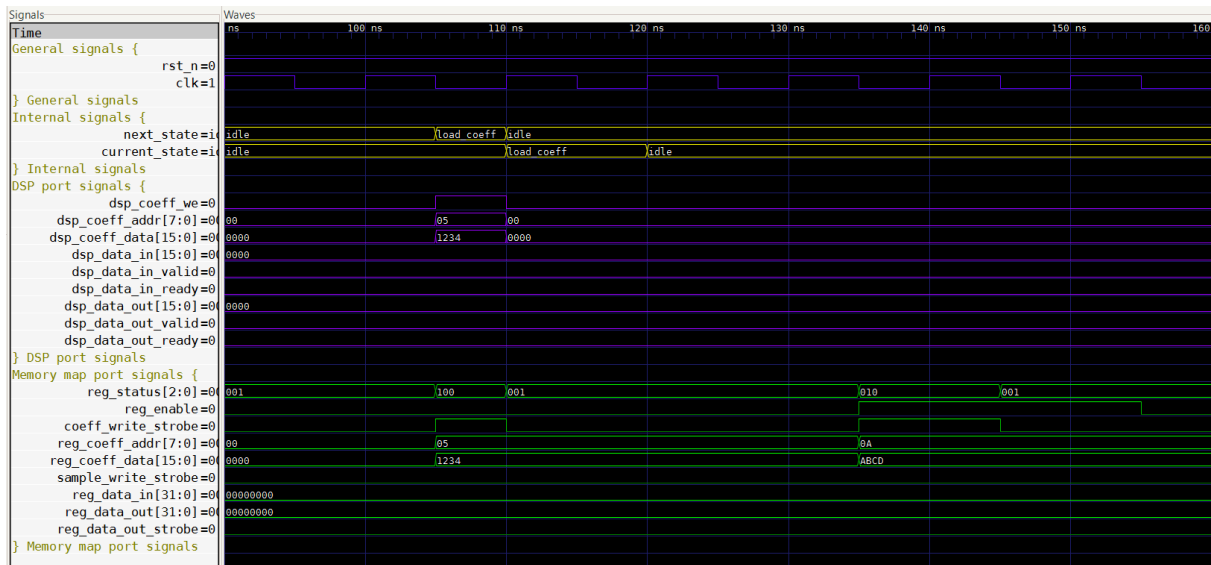


Figure 14 – Test 3 and 4

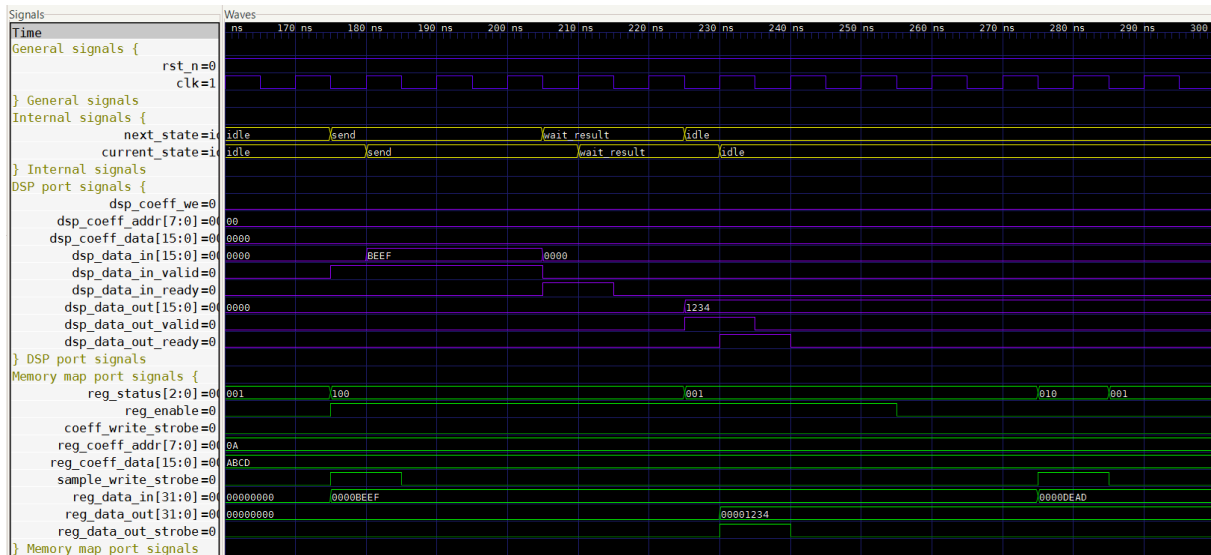


Figure 15 – Test 5 and 6

In test 5, `dsp_data_in_ready` must be asserted when `dsp_data_in_valid` is high. This is an error on the testbench.

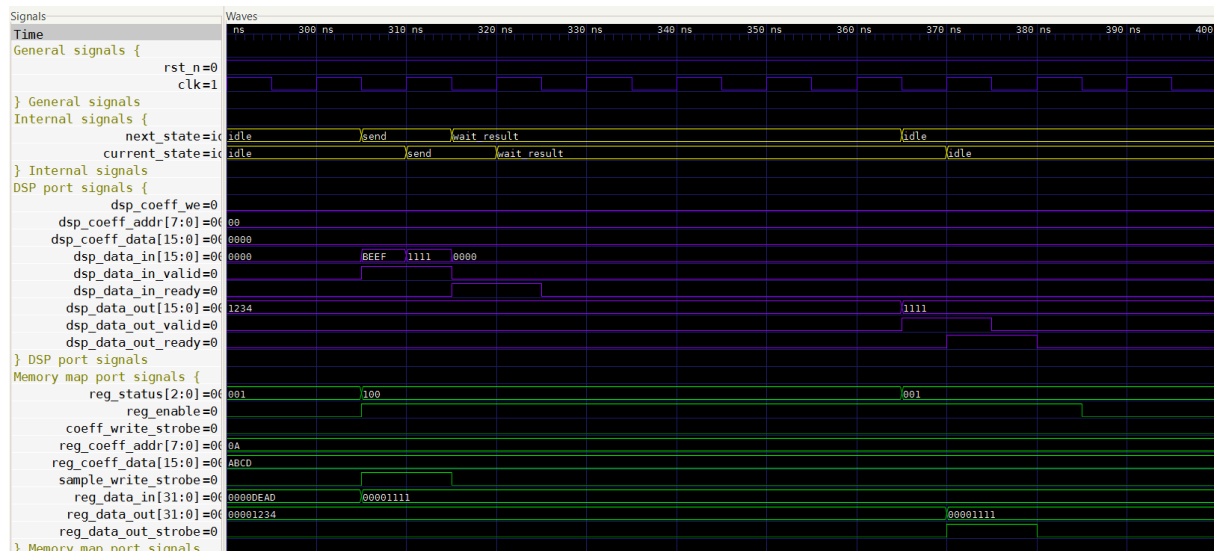


Figure 16 – Test 7

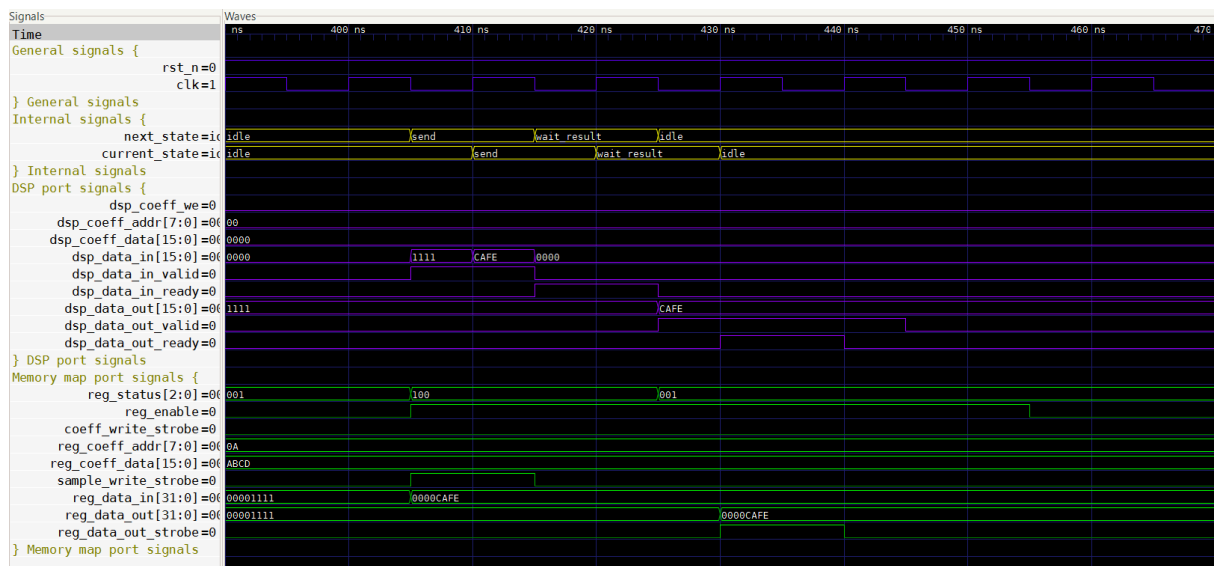


Figure 17 – Test 8

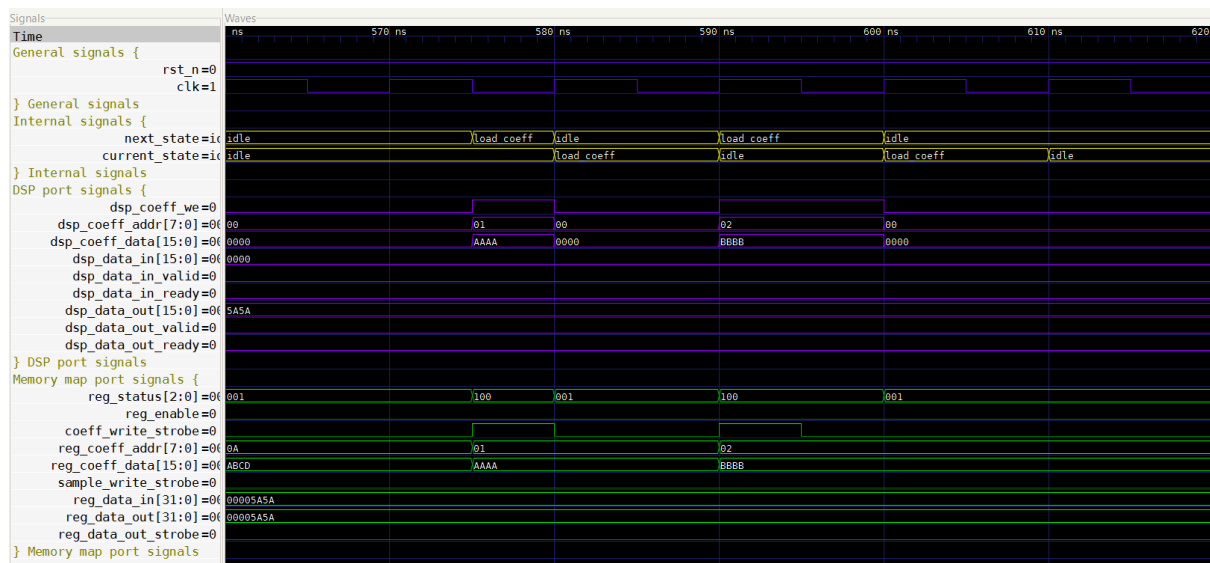


Figure 18 – Test 10